TRACE Tutor

Today's work & learning guide

07 October 2026 • Asia/Dhaka

Purpose: understand what changed, why the fixes work, which commands belong on your laptop or server, and how to verify the result.

What we worked on

Area	Result at the end of this session
Settings page	Layout fix implemented locally; build and lint passed. Live deployment and visual confirmation remain pending.
Code execution	Docker permissions and wrapper compatibility repaired. Your server reported Docker containment active and execution allowed; PM2 was online.
Profile picture	SERVE_MEDIA=1 added in nano. The last screenshot still showed unsaved changes; save, restart and visual confirmation remain pending.

Your deployment

The live site is https://trace.musfiqurtuhin.me. The server is Ubuntu at 89.167.59.65, with the application in /srv/trace-tutor. Your SSH account is shihab.

The backend runs under PM2 using Waitress on 127.0.0.1:3010. nginx provides HTTPS. Student programs run in separate Docker containers through a restricted wrapper.

The key lesson

A change on your Windows laptop does not automatically update the Ubuntu server. Code must be transferred or deployed, configuration saved, and the correct server process restarted.

This document contains no API keys, database passwords, or secret values. Screenshots containing .env secrets are intentionally omitted.

Settings page: layout and scrolling

The symptom

The profile/settings screenshots showed cards cut off, the sidebar moving upward, and a large blank area below the visible content. This pointed to scrolling and sizing in the app shell.

The code changes

In frontend/src/components/Layout/AppShell.jsx, the outer shell changed from a viewport-height block to a fixed viewport shell:

```
Before: h-screen w-screen overflow-hidden
After:  fixed inset-0 overflow-hidden
```

The main content gained overscroll-contain. This keeps a scroll gesture at the edge of the content from passing into outer scrolling.

In frontend/src/pages/Profile.jsx, the profile's root element gained shrink-0. This prevents a long settings form from being compressed by its flex parent.

Checks run locally

```
cd frontend
npm run build
npm run lint
```

Both passed. The first build attempt failed because the filesystem sandbox denied an ancestor-directory lookup; the same build succeeded outside that restriction. No connected browser was available for visual QA.

What still needs deployment

Only the two backend runner files were shown being uploaded to the live server. There is no evidence that the frontend layout changes have been deployed yet.

For a future frontend deployment, transfer those two changed frontend source files using your normal deployment method. From the server's frontend directory, build with:

```
cd /srv/trace-tutor/frontend
npm run build
```

Then refresh the live profile page. Keep the sidebar/header visible and scroll to the final settings card. The server build and visual check above are follow-up steps, not commands confirmed executed today.

Docker sandbox: why execution failed

The original errors

```
cat: probe.txt: Permission denied
available containment (rlimit) is below the required docker
```

These were connected. Docker failed its workspace probe, so automatic detection found only rlimit. The production policy correctly refused the weaker tier.

Ownership mismatch

Python creates temporary work directories with permission mode 0700. Only the directory's owner can access them. The backend owned the directory, but the runner image used UID/GID 65532:65532.

If those user IDs differ, the runner cannot enter the mounted directory. Source files can also become unreadable under a restrictive umask. The compiler needs permission to create its executable there, not just read a probe.

The secure fix

- A non-root Linux backend runs the container with its effective UID and GID, explicitly using --user UID:GID.
- A root backend hands private workdirs and source/probe files to runner UID/GID 65532. Student code never runs as root.
- Workdirs stay private at 0700. Source and probe files use 0600. We did not make them world-writable.
- A random-token probe checks container reads, compiler-step writes, and whether the backend sees the written result.

Host paths must mean the same directory

A backend container and the host Docker daemon can have different filesystems. A Linux systemd service using PrivateTmp=true also has a private /tmp. A bind source is interpreted by the daemon, not by the client process.

The optional shared workspace settings handle this distinction. The provided systemd unit was updated to use /run/trace-tutor-work, but your actual PM2 deployment does not use that unit.

Docker containment remains enforced: no container network, dropped capabilities, no-new-privileges, resource caps, and read-only mounts/root during program execution.

Files changed and wrapper compatibility

File	What changed
frontend/src/components/Layout/AppShell.jsx	Fixed viewport shell; contained content scrolling.
frontend/src/pages/Profile.jsx	Prevented the settings form from shrinking.
backend/tutor/sandbox.py	Workspace ownership; explicit non-root UID/GID; host path mapping; read/write probe; optional volume compatibility.
backend/tutor/runner.py	Uses workspace helpers for temp dirs and C/C++/Python sources; clearer status errors.
backend/tutor/test_sandbox.py	Permissions, mapping, mounts and C regression tests; deterministic reprobe test clock.
backend/trace_backend/settings.py	Reads workspace roots and mount-style configuration.
backend/.env.example	Documents shared workspaces and restricted-wrapper compatibility.
deploy/linux/trace-tutor.service	Shared runtime directory while preserving PrivateTmp; relevant only to the optional systemd deployment.

The first server upload exposed a second issue

```
trace-docker: refused: flag not allowed: --mount
```

You inspected the existing wrapper and shim with these read-only server commands:

```
cat /usr/local/bin/trace-docker
cat ~/bin/docker
```

Your server uses /usr/local/bin/trace-docker, reached through ~/bin/docker. It allows --volume and non-root --user, but rejects --mount. The new default mount syntax was therefore incompatible with this wrapper.

We added CODE_SANDBOX_MOUNT_STYLE=volume as an explicit compatibility setting. It requires a configured, existing workspace at the same absolute host/backend path. The read/write probe remains mandatory. The default elsewhere stays bind.

The wrapper was not changed, and you were not added to the Docker group. Environment fallbacks let the existing server settings module read the new options without uploading the whole settings file.

sandbox.Dockerfile required no code change. Its image was built locally. The server already had the runner image, so this Python-only repair required no server image rebuild.

Deployment: Windows commands and SSH

Recognize where you are

Terminal prompt	Machine and shell
PS C:\Users\…\backend>	Your Windows laptop; PowerShell commands.
shihab@…:/srv/trace-tutor/backend$	Your Ubuntu server; Bash/Linux commands.

The active Python environment shown in a terminal does not tell you which machine you are on. Read the prompt and directory before executing a command.

Upload the repaired runner files

If you are inside SSH, exit to PowerShell first. The upload commands below assume your Windows terminal is in the project's backend folder.

```
exit
```

```
scp -i "$env:USERPROFILE/.ssh/trace_tutor" `
  .\tutor\sandbox.py .\tutor\runner.py `
  shihab@89.167.59.65:/srv/trace-tutor/backend/tutor/
```

scp copies files securely over SSH. -i selects your SSH private key. The destination is the server's tutor directory. You repeated this upload after we added wrapper compatibility.

Connect to the server

```
ssh -i "$env:USERPROFILE/.ssh/trace_tutor" `
  shihab@89.167.59.65
```

ssh opens a terminal on the remote server. It does not upload source files by itself. Commands entered after connecting run on Ubuntu until you type exit.

Why local runserver did not fix the live site

VS Code's Run Code action first tried /usr/bin/env, a Linux path unavailable on Windows. Starting Django from the local terminal then worked:

```
cd backend
python manage.py runserver
```

That started a local development server at 127.0.0.1:8000. It was useful only for local testing. Your public site continued to use PM2/Waitress on the Ubuntu server. For that deployment, restart trace-tutor through PM2.

Deployment: workspace, .env and restart

Prepare the shared workspace on Ubuntu

```
mkdir -p /srv/trace-tutor/sandbox-work
chmod 700 /srv/trace-tutor/sandbox-work
cd /srv/trace-tutor/backend
nano .env
```

mkdir -p creates the directory if needed. chmod 700 lets its owner read, write and enter it, while denying access to other users. nano opens the deployment configuration.

Final sandbox configuration

Add or update each key once; avoid duplicate entries. Leave unrelated settings intact.

```
CODE_SANDBOX=docker
CODE_SANDBOX_REQUIRED=1
CODE_SANDBOX_MIN_TIER=docker
CODE_SANDBOX_MOUNT_STYLE=volume
CODE_SANDBOX_WORKDIR_ROOT=/srv/trace-tutor/sandbox-work
CODE_SANDBOX_HOST_WORKDIR_ROOT=/srv/trace-tutor/sandbox-work
```

docker selects the container runner. REQUIRED=1 enforces containment, MIN_TIER=docker rejects rlimit, and volume selects syntax allowed by your wrapper. Both roots point to the same existing directory on this host deployment.

Save in nano

An asterisk beside .env means changes are unsaved. Press Ctrl+O, Enter to write the file, then Ctrl+X to exit. Alt+/ moves to the end of the file. Saving a file does not restart an already-running backend.

Server verification and test evidence

Check the sandbox and restart the application

```
../venv/bin/python manage.py shell \
  -c "from tutor import sandbox; print(sandbox.status())"
pm2 restart trace-tutor --update-env
```

The first command loads Django settings and runs the actual containment probe. ../venv/bin/python selects the server's Python environment. PM2 then restarts the deployed backend; --update-env refreshes PM2 environment values.

Your server screenshot confirmed:

```
'tier': 'docker'
'isolated': True
'enforced': True
'required_tier': 'docker'
'allowed': True
'detail': None
```

PM2 also reported trace-tutor online. This confirms the server-side probe and restart, not the final browser C execution result.

Local verification evidence

Test environment / revision	Result
Initial Windows host backend → Docker	50 tests: 42 passed, 8 skipped.
Linux root backend container → host daemon	50 tests passed; runner UID/GID 65532:65532.
Linux backend UID/GID 1001:1002 → host daemon	50 tests passed; restrictive umask 077.
Final volume compatibility on Windows	54 tests: 50 passed, 4 POSIX-only tests skipped.
Final targeted tests with older settings	25 tests passed; environment fallback verified.

These are separate runs/revisions, not a count of unique tests. The full database-backed application suite was not run. Linux isolation checks included blocked network access, denied filesystem writes, non-root execution, timeouts, memory exhaustion and fork-bomb containment.

C program: execution and live-site check

The requested program

```
#include <stdio.h>
int main() {
    int n, sum = 0;
    scanf("%d", &n);
    for (int i = 1; i <= n; i++) sum += i;
    printf("%d\n", sum);
    return 0;
}
```

```
Input: 5
Expected output: 15
```

The program sums 1 + 2 + 3 + 4 + 5. Local Docker runs returned SUCCESS, output 15, and exit code 0. The original Linux permission error was reproduced and then resolved. Incorrect host paths were rejected.

Local verification commands

The production runner image was built on Windows from the repository root:

```
docker desktop start
docker build -f backend/tutor/sandbox.Dockerfile `
  -t trace-tutor-runner:1 backend/tutor
```

A trailing backtick continues a PowerShell command onto the next line. This build belongs on a host with Docker build access; your server wrapper does not grant that access.

The final compatibility suite ran locally using your installed Python environment:

```
E:/projects/anaconda/envs/trace_tutor/python.exe `
  docs/sandbox-verification/verify_volume_compat.py
```

The final live-site check

Refresh the public site with Ctrl+Shift+R, open the C workspace, enter the code, give input 5 and click Run Code. Verify output 15. The conversation does not yet include that live browser result.

Profile picture: production media serving

The symptom and likely cause

Your uploaded avatar displayed a broken-image icon and the name text. The application generated an avatar URL, but the browser could not load the image. We inspected avatar generation, MEDIA_ROOT, MEDIA_URL, the media route and the serving flag.

The code defaults SERVE_MEDIA to DEBUG. Your server has DEBUG=0, so Django's media view returns 404 unless SERVE_MEDIA=1 or another file server serves /media/. That was the likely cause; the actual image HTTP response was not captured.

The configuration change

```
cd /srv/trace-tutor/backend
nano .env
```

Add or update:

```
SERVE_MEDIA=1
```

Save with Ctrl+O, Enter, exit with Ctrl+X, then restart:

```
pm2 restart trace-tutor --update-env
```

Refresh the live page with Ctrl+Shift+R. This configuration permits the existing media route to serve uploaded files under backend/media/. No avatar code changes or image deletion were needed.

What remains to verify

The last screenshot showed SERVE_MEDIA=1 in nano with an unsaved marker. We confirmed the line was correct and instructed you to save and restart, but no subsequent screenshot confirms that the avatar loaded.

If it still fails, open the image in a new tab and note the URL and HTTP/error result. An incorrect URL, missing file, file permissions, or nginx handling can also cause broken images. Do not assume that re-uploading alone fixes a serving configuration problem.

Configuration safety

Keep DEBUG=0 in production. Serving avatars does not require turning on Django debug mode. Your .env screenshots exposed credentials; arrange rotation with the server owner. This learning document intentionally does not copy those secret values.

Your reusable checklist and command glossary

Repeat this workflow for a future fix

- Inspect the symptom and logs. Decide whether the cause is application code, environment configuration, file permissions, or deployment.
- Make the smallest relevant local change. Run appropriate checks and preserve evidence of the result.
- Confirm the target machine. Upload only the intended changed files; do not replace the server .env with your laptop .env.
- Save configuration changes. Check the service's Python environment, workspace path, and current user permissions.
- Restart the correct deployed process. For this server, use PM2/Waitress, not manage.py runserver or the optional trace-tutor systemd unit.
- Verify server health and the actual browser action. A process being online is not proof that every feature works.

Useful server commands

```
pm2 list
pm2 logs trace-tutor --lines 50 --nostream
command -v docker
id
ls -ld /srv/trace-tutor/sandbox-work
```

These are reference commands for future diagnosis; they are not all shown executed on your server during this session. The Docker command should resolve through your user shim, which uses the locked wrapper.

Term	Meaning
UID / GID	Numeric user/group IDs used to evaluate Linux file access.
Bind mount	A host directory exposed at a path such as /work in a container.
Docker versus rlimit	Docker isolates network/filesystem; rlimit primarily caps resources.
Probe versus execution	A probe checks readiness; the live Run Code action confirms the full feature.
Build versus restart	A frontend build creates assets; a backend restart reloads Python/configuration.

Sources: session screenshots, project diffs, sandbox verification logs, and the supplied server guide. Confirmed actions and recommended steps are identified separately.